

# UDRIVE · MANUAL TÉCNICO

## Arquitectura, métodos principales, diagramas de flujo y mantenimiento

Alex Ricardo Castañeda Rodríguez · Carné 202300476 · Sección B

**Alcance**

Este documento describe la aplicación principal en src/main, que conserva MainFrame.form y Distance.form de NetBeans e integra la lógica pendiente. legacy/2024 es una copia histórica, no la aplicación activa.

## Contenido

| Sección                    | Qué documenta                                           |
|----------------------------|---------------------------------------------------------|
| 1. Solución y arquitectura | Capas, diagrama de clases y responsabilidades.          |
| 2. Modelo y reglas         | Entidades, estados y consumo de combustible.            |
| 3. Rutas                   | CSV, validación, importación y edición de distancia.    |
| 4. Preparación e inicio    | Asignación de piloto/vehículo y creación del hilo.      |
| 5. Avance y retorno        | Métodos de Journey, recarga y finalización.             |
| 6. Persistencia y UI       | Serialización, restauración y sincronización con Swing. |
| 7. Compilación y pruebas   | Estructura, comandos y cobertura.                       |

## Correspondencia con el enunciado

El enunciado solicita un manual técnico con descripción general, diagrama de clases y diagramas de flujo de los métodos principales. Las secciones siguientes cumplen esos tres puntos y añaden contratos y casos de error tomados del código actual.

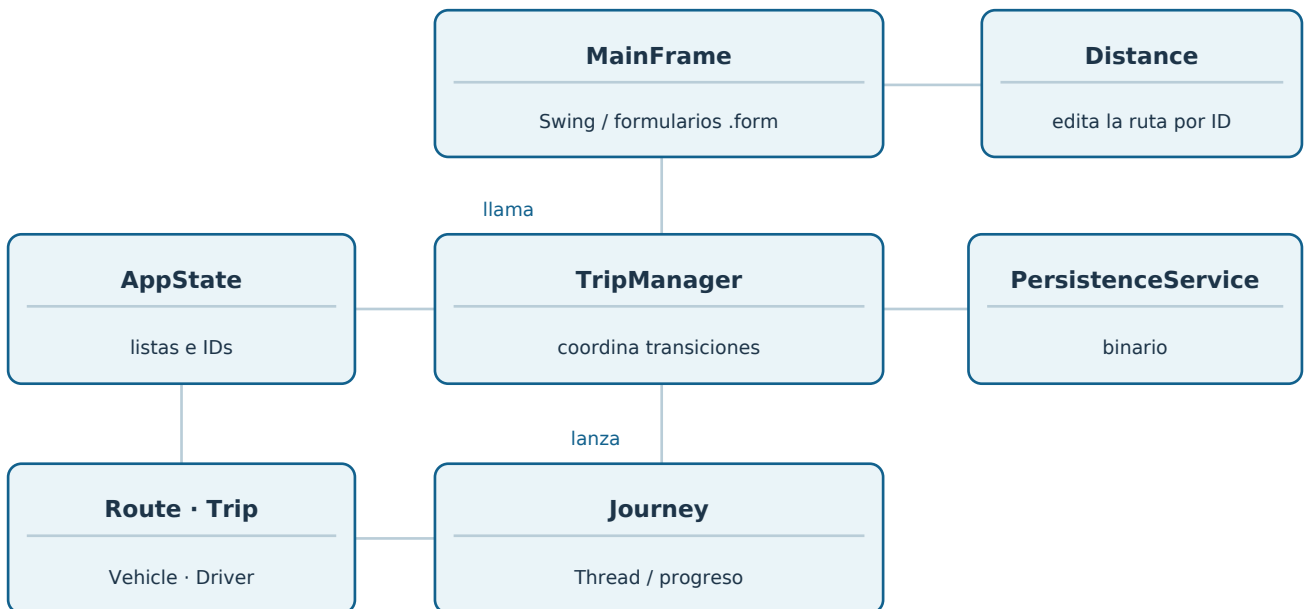
## Puntos de entrada para revisar el código

| Archivo                          | Dónde comenzar                                     |
|----------------------------------|----------------------------------------------------|
| src/main/Main.java               | Carga el estado y muestra MainFrame.               |
| src/main/TripManager.java        | Concentra las operaciones del ciclo de viaje.      |
| src/main/Journey.java            | Implementa el avance concurrente y el combustible. |
| src/main/PersistenceService.java | Lee y escribe el archivo serializado.              |

## 1. Solución y arquitectura

La interfaz Swing original presenta cuatro vistas dentro de MainFrame: rutas, generación, inicio y seguimiento, e historial. TripManager concentra las transiciones de negocio. AppState guarda los objetos; Journey ejecuta cada viaje en un hilo; CsvRouteImporter y PersistenceService gestionan entrada y salida.

### Diagrama de clases y colaboraciones



Las líneas indican uso o propiedad entre componentes; no representan herencia. Journey sí hereda de Thread. Los formularios .form se conservan editables en NetBeans y se compilan con AbsoluteLayout.

| Capa           | Clases                                                          | Responsabilidad                            |
|----------------|-----------------------------------------------------------------|--------------------------------------------|
| Presentación   | Main, MainFrame, Distance                                       | Arranque, eventos Swing, tablas y diálogo. |
| Aplicación     | TripManager, Journey                                            | Reglas de transición, hilos y callbacks.   |
| Dominio        | AppState, Route, Trip, Driver, Vehicle, VehicleType, TripStatus | Estado serializable y restricciones.       |
| Entrada/salida | CsvRouteImporter, PersistenceService                            | CSV UTF-8 y archivo binario.               |

## 2. Modelo de datos y reglas de negocio

| Clase    | Datos clave / contrato                                                                                              |
|----------|---------------------------------------------------------------------------------------------------------------------|
| Route    | id, start, end, distance. connects() acepta el mismo par de extremos en ambos sentidos.                             |
| Trip     | ID, ruta, origen/destino, vehículo/piloto, fechas, estado, progreso por etapa, distancia total y combustible total. |
| Vehicle  | Tipo, número de unidad, combustible restante y viaje asignado. consume() no permite saldo negativo.                 |
| Driver   | ID, nombre y viaje asignado; solo uno a la vez.                                                                     |
| AppState | Listas de entidades y contadores para nuevos IDs; inicia tres pilotos y nueve vehículos con tanque lleno.           |

### Máquina de estados de Trip

```
PREPARED → OUTBOUND → WAITING_RETURN → RETURNING → COMPLETED
                        |
                        |   Finish here → COMPLETED
                        |   └─ OUT_OF_FUEL_OUTBOUND → Refuel → OUTBOUND
                        |   RETURNING → OUT_OF_FUEL_RETURN
                        |           → Refuel → RETURNING
```

WAITING\_RETURN mantiene recursos reservados. COMPLETED libera piloto y vehículo. beginReturn() reinicia el progreso de la etapa, pero conserva la distancia total y el combustible acumulado.

### Parámetros de combustible

| Tipo        | gal/km | Capacidad (gal) |
|-------------|--------|-----------------|
| Motocicleta | 0.10   | 6               |
| Estándar    | 0.30   | 10              |
| Premium     | 0.45   | 12              |

Por cada tramo, consumo = distancia del paso × tasa del tipo. Por ejemplo, 45 km en un vehículo estándar consumen 13.5 galones: se requerirá recarga antes de terminar con un tanque de 10 galones.

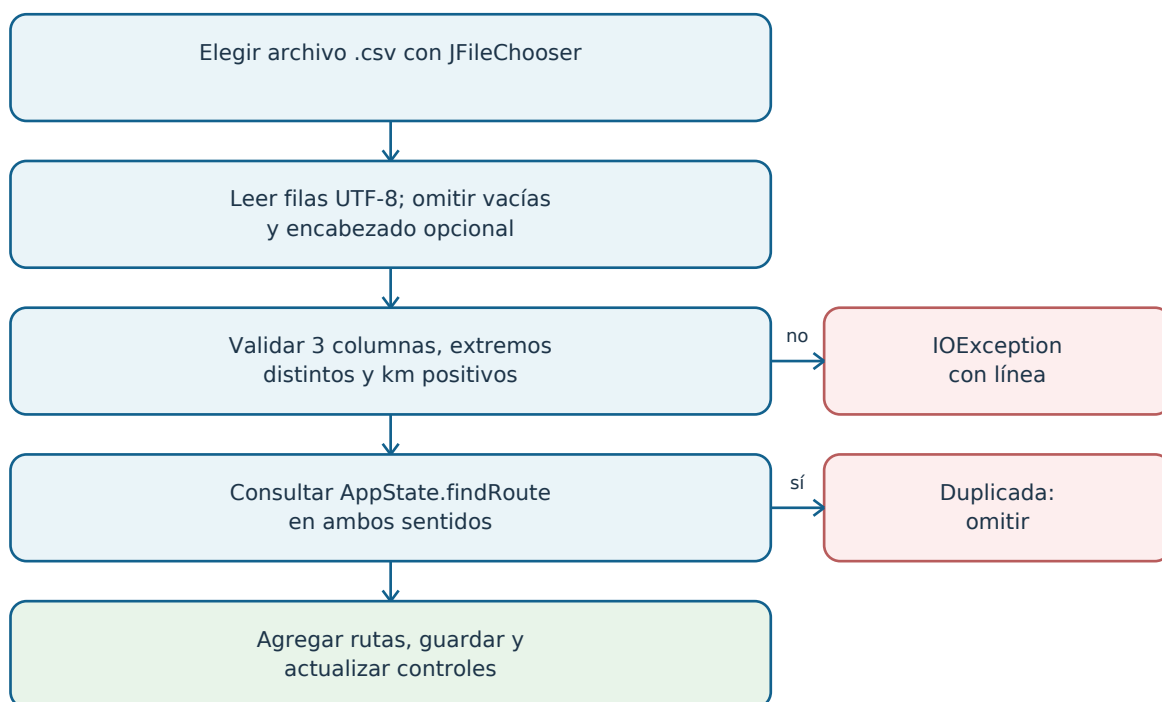
### Invariantes que deben mantenerse

| Regla        | Implementación                                                                                         |
|--------------|--------------------------------------------------------------------------------------------------------|
| Identidad    | AppState incrementa nextRouteId y nextTripId; no reutiliza IDs al terminar viajes.                     |
| Exclusividad | Un piloto y un vehículo tienen a lo sumo un assignedTripId activo.                                     |
| Distancia    | Trip guarda una copia de routeDistanceKm al crearse; editar Route no cambia viajes ya creados.         |
| Acumulados   | beginReturn() reinicia solo el progreso de etapa; totalDistanceKm y fuelConsumed continúan acumulando. |

### 3. Funciones principales: rutas

| Método                                | Entrada y resultado                                                                                    |
|---------------------------------------|--------------------------------------------------------------------------------------------------------|
| CsvRouteImporter.read(Path)           | Lee UTF-8 y devuelve List<RouteData>. Lanza IOException con número de línea si el formato es inválido. |
| MainFrame.chooseCSVFile()             | Abre JFileChooser filtrado a .csv; entrega el archivo a importCsv().                                   |
| MainFrame.importCsv()                 | Agrega rutas nuevas, omite duplicadas, guarda el estado y actualiza tabla/listas.                      |
| MainFrame.updateRouteDistance(id, km) | Devuelve false si el ID no existe o $km \leq 0$ ; de lo contrario edita, guarda y refresca.            |

#### Diagrama de flujo: CsvRouteImporter.read + importCsv



La validación termina antes de agregar rutas porque read() devuelve la lista completa. El archivo de ejemplo está en examples/rutas.csv. La edición de distancia se hace por ID, no seleccionando una fila.

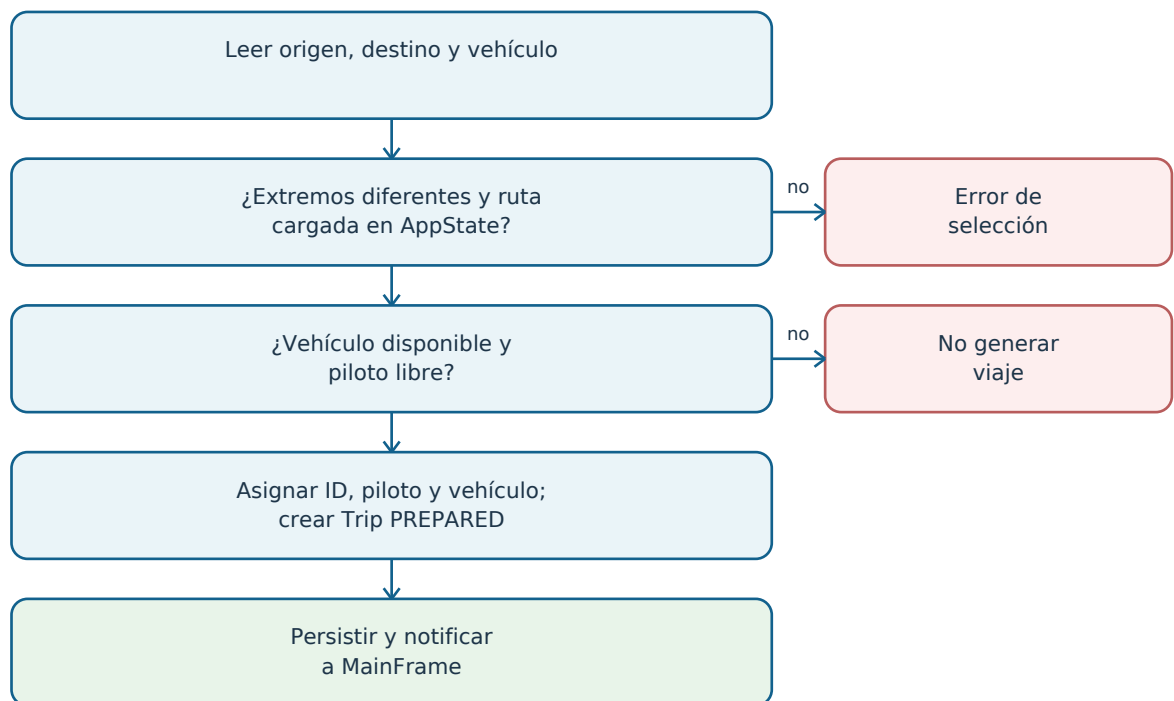
#### Casos límite de la importación

| Entrada                         | Respuesta                                                                                     |
|---------------------------------|-----------------------------------------------------------------------------------------------|
| Fila vacía                      | Se ignora.                                                                                    |
| Encabezado Inicio,Fin,Distancia | Se omite solo si está en la primera línea; los nombres se comparan sin distinguir mayúsculas. |
| Ruta repetida                   | No se agrega una segunda vez, incluso si los extremos vienen invertidos.                      |
| Archivo sin rutas válidas       | read() lanza IOException y la interfaz muestra un error.                                      |

## 4. Funciones principales: preparar e iniciar

| Método                                                           | Precondición, efecto y salida                                                                                             |
|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| MainFrame.generateTripFromForm()                                 | Lee los desplegables, busca el vehículo disponible y solicita a TripManager la creación; muestra error o abre Trip start. |
| TripManager.createTrip(origin, destination, vehicle)             | Exige extremos distintos, ruta existente, vehículo libre y piloto libre. Crea Trip, guarda y notifica la UI.              |
| AppState.createTrip(route, origin, destination, vehicle, driver) | Genera ID, reserva ambos recursos y añade el viaje PREPARED a la lista.                                                   |
| TripManager.startTrip(trip)                                      | Solo desde PREPARED: marca hora de inicio, pasa a OUTBOUND y lanza Journey.                                               |
| TripManager.startAll()                                           | Recorre los viajes y aplica startTrip a cada PREPARED.                                                                    |

### Diagrama de flujo: TripManager.createTrip



El límite efectivo de tres viajes activos proviene de los tres pilotos. El vehículo se reserva al preparar el viaje, no al pulsar Start.

### Errores y postcondiciones

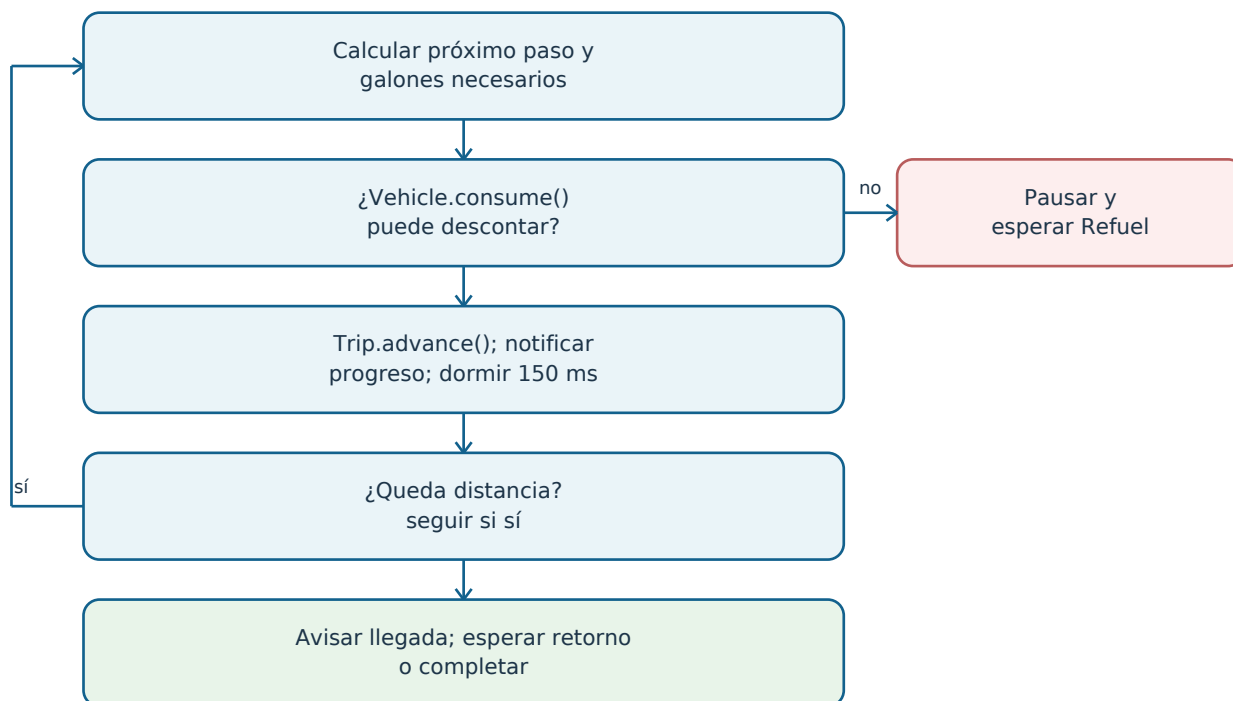
`createTrip()` produce `IllegalArgumentException` si los extremos son iguales, falta la ruta o el vehículo no está libre; produce `IllegalStateException` cuando no hay piloto disponible. `MainFrame` captura esos errores y muestra un diálogo. Si la creación tiene éxito, el nuevo Trip está en PREPARED y el piloto y vehículo quedan reservados.

`startTrip()` no vuelve a arrancar un viaje que ya está en curso. `startAll()` aplica la misma operación a cada viaje preparado, por lo que el inicio individual y el conjunto comparten la misma regla de estado.

## 5. Funciones principales: avance, combustible y retorno

| Método                                | Comportamiento                                                                                   |
|---------------------------------------|--------------------------------------------------------------------------------------------------|
| Journey.run()                         | Cada 150 ms intenta avanzar hasta 1 km; calcula consumo, actualiza Trip y avisa progreso.        |
| Vehicle.consume(gallons)              | Devuelve false si el tanque no alcanza para el próximo paso; no descuenta combustible.           |
| TripManager.refuel(trip)              | Solo en OUT_OF_FUEL_OUTBOUND/RETURN; llena el tanque y despierta al hilo con fuelAvailable().    |
| TripManager.startReturn(trip)         | Solo en WAITING_RETURN; reinicia progreso de etapa, cambia a RETURNING y lanza un Journey nuevo. |
| TripManager.finishAtDestination(trip) | Si espera en destino, completa allí, sin recorrido de vuelta.                                    |
| TripManager.complete(trip)            | Marca COMPLETED, libera recursos, elimina el hilo activo, guarda y notifica.                     |

### Diagrama de flujo: Journey.run



Si el tanque queda corto, el hilo espera en fuelMonitor. En ida pasa a OUT\_OF\_FUEL\_OUTBOUND; en retorno a OUT\_OF\_FUEL\_RETURN. La interfaz carga el GIF orientado al lado correspondiente.

### Ejecución concurrente y callbacks

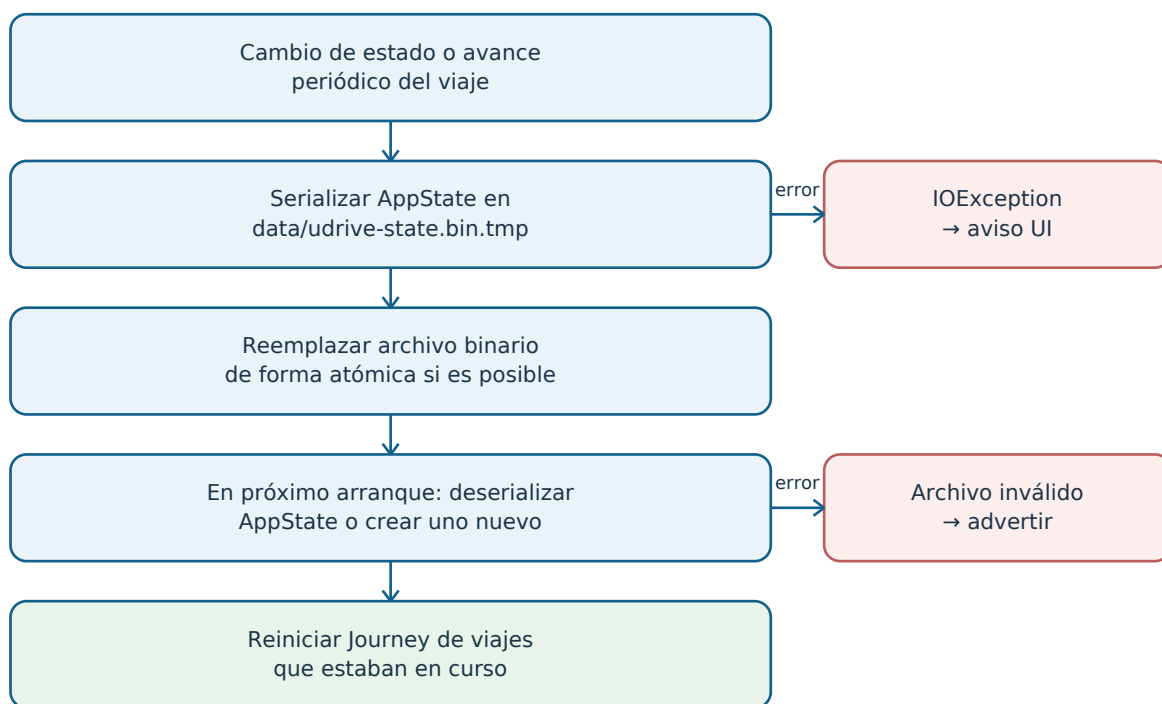
Cada Journey es un hilo daemon identificado por viaje. Cuando avanza llama a onProgress; al faltar combustible a onFuelEmpty; al llegar a onArrived. TripManager traduce esos eventos en guardado, cambio de estado y actualización de la interfaz. El avance es de hasta 1 km por tick de 150 ms; es una simulación proporcional a la distancia, no tiempo real de carretera.

Ejemplo: una ruta de 2 km necesita dos pasos y consume 0.2 galones en motocicleta. Si se retorna, beginReturn() reinicia la etapa y la trayectoria total llega a 4 km; el consumo total será 0.4 galones si no hubo otras variaciones.

## 6. Persistencia y concurrencia

| Método                            | Responsabilidad                                                                                     |
|-----------------------------------|-----------------------------------------------------------------------------------------------------|
| PersistenceService.load()         | Si el archivo no existe, crea AppState. Si existe, deserializa y valida que el objeto sea AppState. |
| PersistenceService.save(state)    | Escribe a .tmp y lo reemplaza con move atómico cuando está disponible; usa ObjectOutputStream.      |
| TripManager.restoreRunningTrips() | Al iniciar, relanza Journey para estados OUTBOUND, RETURNING y pausas de combustible.               |
| TripManager.shutdown()            | Cancela hilos activos y guarda el estado final al cerrar.                                           |

### Diagrama de flujo: guardado y restauración



Los métodos que modifican el estado compartido están sincronizados en TripManager, AppState y entidades relevantes. runningJourneys es un ConcurrentHashMap por ID. Las notificaciones provenientes de Journey llegan a Swing mediante SwingUtilities.invokeLater para pintar en el hilo de eventos.

Importante: la ruta de datos es relativa al directorio desde el que se ejecuta Java. El CSV es solo entrada; el estado persistente solicitado por el enunciado es binario.

### Cuándo se guarda

| Evento              | Disparador                                                                               |
|---------------------|------------------------------------------------------------------------------------------|
| Rutas / preparación | Tras importar, editar distancia o crear un viaje.                                        |
| Transiciones        | Al iniciar, llegar, recargar, regresar, completar o cerrar.                              |
| Progreso            | Periódicamente durante Journey, aproximadamente cada 5 km recorridos en la etapa.        |
| Error de escritura  | TripManager notifica a MainFrame; se muestra un diálogo y el estado en memoria continúa. |

## 7. Interfaz, compilación y verificación

MainFrame.refreshRoutesTable() reconstruye la tabla de rutas; refreshSelections() actualiza orígenes, destinos, flota y pilotos disponibles; refreshTrips() sitúa los GIF y habilita Start, Refuel, Return y Finish here según TripStatus; refreshHistory() muestra solo viajes COMPLETED.

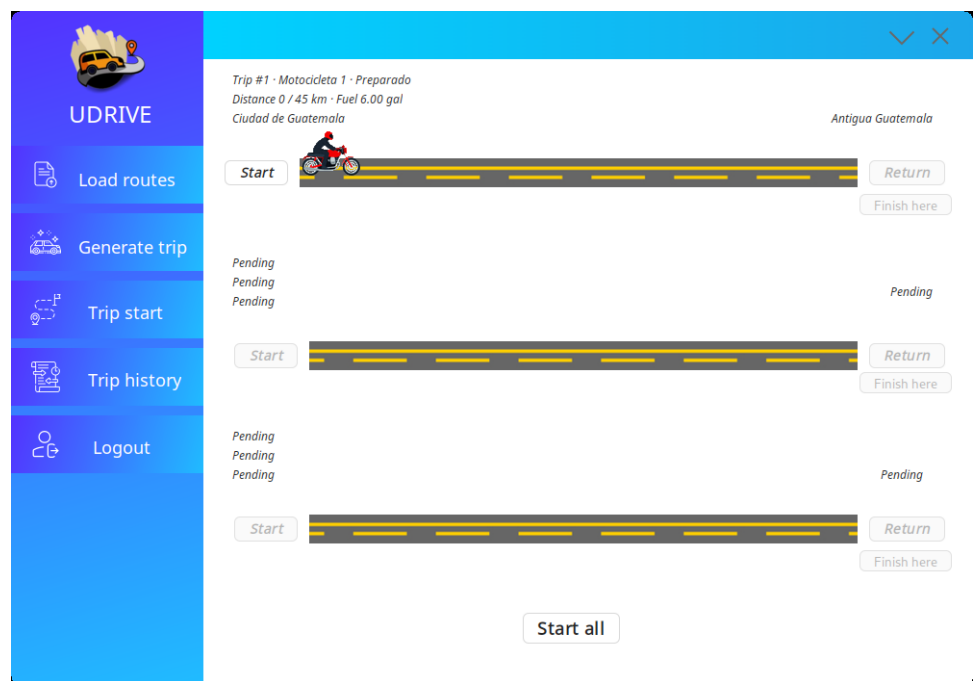


Figura 1. Pantalla de viajes del formulario original; la lógica actual se conecta a estos controles.

### Estructura y comandos

```
src/main/      Java y formularios NetBeans
src/vehicles/  imágenes y GIF de ida/retorno
test/main/    pruebas JUnit 5
docs/         enunciado, manuales y capturas
./mvnw clean package
./mvnw test
java -jar target/UDrive.jar
```

Maven usa Java 21, FlatLaf, NetBeans AbsoluteLayout y Shade para producir el JAR ejecutable. target/ es generado y no debe subirse al repositorio. Si se modifica un formulario en NetBeans, mantenga sincronizados su .java y .form.

### Pruebas automatizadas existentes

| Clase de prueba      | Verifica                                                    |
|----------------------|-------------------------------------------------------------|
| CsvRouteImporterTest | Encabezado, filas incompletas y distancias no positivas.    |
| AppStateTest         | Flota inicial, reserva/liberación y serialización.          |
| JourneyTest          | Avance, consumo y pausa/reanudación por combustible.        |
| TripManagerTest      | Completar en destino y retorno con recursos liberados.      |
| VehicleIconTest      | GIF de retorno para las nueve unidades y tamaños de lienzo. |